Prompt Optimizers Can Make Tool-Using Agents Worse: An Equal-Budget Study and a Gated Alternative

Kumar Ashish
Harvard Extension School, Harvard University
kua613@g.harvard.edu

Abstract

Prompt and context optimizers such as GEPA, MIPROv2 and ACE improve language-model systems by rewriting their instructions from execution feedback, and are increasingly applied to tool-using agents. Their reported gains come from comparisons run by their authors, on their own tasks, usually with one run per configuration. We compare them under one harness: the same agent (gpt-oss-20b), reflection model, practice tasks, episode rendering and rollout budget, a frozen test split, three seeds, and a protocol fixed before the main runs. On τ^2-bench airline and retail, no optimizer reliably improves the agent within 600 rollouts, while the test score of the unchanged agent alone varies from 0.38 to 0.54 between scorings. The competitors often ship a worse agent: GEPA's and MIPROv2's shipped agents score below every scoring of the unchanged agent in 4/9 and 2/9 of their runs, one of them with a rewritten prompt that told the agent to type its tool calls as JSON text. We introduce CLADE, which edits one component at a time (a rule, a prompt section or a tool description), screens proposed fixes for task-specific content, and promotes a change only if it wins on held-out tasks without a per-task drop larger than run-to-run noise. CLADE never shipped a harmful agent (0/9 runs), was ahead of every competitor on retail (significantly against one of four after correction, marginally against a second), improved the agent on a development benchmark by +0.13 [+0.01, +0.27], and was the cheapest and fastest method. Its own gain on τ^2-bench is small and not significant, and our ablations do not isolate a single component. We release the harness, protocol and all runs.

1 Introduction

When an agent's weights cannot be changed (a closed model, or a deployment where fine-tuning is too costly), the remaining lever is what the model is told: its system prompt, any learned rules, and the descriptions of its tools. A family of optimizers edits these automatically from the agent's own executions. MIPROv2 proposes and searches over instructions (Opsahl-Ong et al., 2024); TextGrad backpropagates textual critiques (Yuksekgonul et al., 2025); GEPA reflects on trajectories and evolves prompts with a Pareto archive (Agrawal et al., 2026); ACE grows a playbook of strategies from reflections (Zhang et al., 2026b). These methods report gains of several to tens of points, and some ship in widely used frameworks (Khattab et al., 2024).

Two features of tool-using agents make those gains hard to transfer. First, agents are stochastic: the same agent on the same task can pass on one run and fail on the next, so a candidate prompt's score on a small validation set is noisy. Second, practice tasks are few: a team with a working agent rarely has hundreds of verified tasks. An optimizer that selects the best of many candidates on a noisy, small validation set will often select noise, and the prompt it ships can be worse than the one it started from. For a practitioner, that is the outcome that matters most: an optimizer that sometimes makes a working agent worse cannot be run unattended.

We test this directly. We build one harness in which every optimizer drives the same agent through the same counted rollout path, reads the same rendering of each episode, uses the same reflection model and practice tasks, and has its shipped agent scored on the same frozen test split, with three seeds and a protocol fixed before the main runs (§4). We also introduce CLADE (§3), an optimizer designed around the failure mode above: it proposes one bounded change at a time, screens proposed fixes before spending episodes on them, and promotes a change only through a held-out gate that tolerates run-to-run noise but no larger per-task drop.

Our findings (§5):

- On τ^2-bench airline and retail with a gpt-oss-20b agent, no optimizer reliably improves the agent within 600 rollouts. The unchanged agent's test score varies by up to 0.16 between scorings, more than most reported gains.
- The competitors frequently ship a worse agent. Across 9 runs each, the shipped agent scored below every scoring of the starting agent in 4/9 runs for GEPA, 4/9 for GEPA with tool descriptions, 2/9 for MIPROv2 and 1/9 for ACE.
- CLADE never shipped a harmful agent (0/9). On retail it is ahead of every competitor, significantly against GEPA with tool descriptions and marginally against MIPROv2 after Holm correction. On our development benchmark it improves the agent by +0.13 [+0.01, +0.27]. It is the cheapest and fastest method per run.
- The protection comes mostly from declining to change the agent: CLADE shipped the starting agent unchanged in three of six τ^2-bench runs, and its own gain over the starting agent on τ^2-bench is small (+0.05 [−0.01, +0.11]). No single component's removal significantly hurts.

2 Related work

Prompt optimization. APE (Zhou et al., 2023) and OPRO (Yang et al., 2024) generate and score candidate instructions; Promptbreeder evolves prompts and the mutation prompts that produce them (Fernando et al., 2024). DSPy (Khattab et al., 2024) compiles language-model programs, with MIPROv2 (Opsahl-Ong et al., 2024) searching instructions and demonstrations by Bayesian optimization and SIMBA (DSPy, 2025) adding rules from contrasting trajectories. TextGrad (Yuksekgonul et al., 2025) and Trace (Cheng et al., 2024) treat textual feedback as a gradient. GEPA (Agrawal et al., 2026) reflects on trajectories in natural language and keeps a Pareto front of candidates, and reports outperforming reinforcement learning with far fewer rollouts. Zhao et al. (2026) compare GEPA and MIPROv2 on τ^2-bench at matched rollouts and find GEPA's prompt-only gains on retail negative; Fu et al. (2026) describe prompt overfitting, where narrow, sample-specific rules accumulate as training loss falls.

Algorithm 1 One CLADE round

plan lanes: the incumbent, then archive parents by clade score, else the incumbent under another lens
for each lane with parent P (in parallel) **do**
    $D \leftarrow$ REFLECT(failed training episodes of P)
    $D \leftarrow$ CRITIC(D, playbook of P)
    $C_P \leftarrow$ CURATE(P, D) ▷ bounded edits
end for
for each candidate c, while the budget allows one more evaluation **do**
    run c n times on every training and held-out task
end for
$c^\star \leftarrow \arg\max_c (\bar{s}_{\text{held}}(c), \bar{s}_{\text{train}}(c))$
if $c^\star$ beats the incumbent **and** $\max_t[s_{\text{inc}}(t) - s_{c^\star}(t)] \leq \max(\delta, 1/n)$ **then**
    promote $c^\star$
end if

Learning from experience. Reflexion (Shinn et al., 2023) turns failures into verbal feedback within a task; ExpeL (Zhao et al., 2024), Agent Workflow Memory (Wang et al., 2025), Dynamic Cheatsheet (Suzgun et al., 2026) and ReasoningBank (Ouyang et al., 2026) accumulate reusable insights across tasks. ACE (Zhang et al., 2026b) curates an evolving playbook through generator, reflector and curator roles with incremental updates. CLADE shares the reflect-and-curate structure; it differs in bounding each change, screening fixes before evaluation, and gating promotion on held-out tasks.

Self-improving agents. The Darwin Gödel Machine (Zhang et al., 2026a) keeps an archive of self-modified agents; the Huxley-Gödel Machine (Wang et al., 2026) chooses which agent to expand by the success of its descendants (clade metaproductivity), which CLADE uses to choose parents among bundle versions. Meta-Harness (Lee et al., 2026) searches over harness code. Concerns about evaluation in this area, including cost, reproducibility and held-out splits, are summarized by Kapoor et al. (2025); our protocol follows their recommendations.

3 CLADE

CLADE optimizes a *bundle*: the agent's system prompt, a list of playbook rules rendered into it, and overrides for tool descriptions. Every candidate is a versioned child of a parent bundle with the diagnosis that produced it, so any promoted change can be traced to the failures it fixed. The model's weights never change.

Reflector. For a parent bundle, the reflector reads its failed episodes on *training* tasks (never held-out

ones): one diagnosis per failing task (up to four) and one cross-cutting diagnosis across tasks, each with the root process mistake and one general fix. Each episode is rendered as its trajectory, final reply and the judge's verdict, trimmed in the middle when long. A fix that quotes code or names an identifier seen in the episode's tool traffic is discarded as task-specific.

Critic. One further model call reviews the surviving fixes and drops those that are hints about one task, not actionable, or already said by the playbook. Fixes are screened before any episode is spent on them.

Curator. Each fix becomes a bounded candidate that changes one component: a single playbook rule (the fix itself, deduplicated against existing rules by a yes/no model call, at most eight rules); one appended prompt section of at most six lines; or a rewrite of one tool's description (at most 400 characters, no identifiers), when a diagnosis concerns tool use. A further candidate combines the round's new rules. Whole-prompt rewrites are never proposed.

Selector and gate. Each candidate is run n times on every training and held-out task. The candidate with the best held-out mean (training mean as tie-break) is promoted only if it beats the incumbent and no task's mean drops by more than $\max(\delta, 1/n)$, with $\delta = 0.34$. With $n = 2$, one extra failed run on a task is tolerated as noise; a larger drop blocks promotion. Training episodes of a promoted candidate become the next round's material for reflection.

Coordinator and archive. Every evaluated version enters an archive. Each round runs up to three lanes in parallel: the incumbent, and the archive's best parents by clade score (mean held-out score of a node's descendants) plus an exploration bonus $c\sqrt{\ln N/(1+k)}$, where N is the archive size and k the node's number of children (Wang et al., 2026). When the archive has no second parent, extra lanes reread the incumbent's failures through a different lens (contrast with a success; tool use only). The run stops after three rounds, after two rounds without a promotion, or when the rollout budget cannot cover another evaluation with room to spare.

4 Experimental setup

Benchmarks. τ^2-bench (Barres et al., 2026) (v1.0.1) simulates customer service: an agent with domain tools and a written policy talks to a simulated user, and is scored 1 only if the final database state matches the expected outcome and, where a task requires it, the agent told the user the required information (airline) or satisfied natural-language assertions checked by a judge (retail). We use the airline and retail domains. Practice tasks are 20 training and 10 validation tasks drawn once from each domain's official train split; the test split is the official one (20 airline, 40 retail tasks), each test task run 4 times. The agent is τ^2-bench's own LLM agent; a bundle replaces only its instruction block and may override tool descriptions, and the domain policy is passed verbatim, so the starting bundle *is* τ^2-bench's default agent. The user simulator and retail's assertion judge are gpt-oss-120b at temperature 0. Our *support desk* is a smaller customer-support environment (9 tools, a refund and cancellation policy, scored by hidden tests on the store's side effects) with 7 training, 3 validation and 20 test tasks, each run 10 times. CLADE was developed on it, so we report it as a development benchmark only.

Models. The agent is gpt-oss-20b (OpenAI, 2025). Every optimizer uses gpt-oss-120b for reflection, proposal and curation.

Competitors. We use each method's official library where one exists and its default configuration sized to the budget: GEPA (gepa 0.1.4, reflective proposer, minibatch 3), once on the system prompt only, as in its experiments, and once with every tool description as an additional component (GEPA+tools), the same search space CLADE edits; MIPROv2 (DSPy 3.4.0, instruction only, trials sized to the budget); and ACE, ported from the official prompts and code (temperature 0, up to three reflection rounds per task, up to five epochs, best epoch on validation). Appendix B lists every setting.

Equal treatment. Every method runs the agent only through the harness, which counts one rollout per episode; an episode lost to the provider is rerun and counted once. All methods read the same rendering of an episode, evaluate validation tasks twice, and receive 600 rollouts per run on τ^2-bench (400 on the support desk). No method sees a test

task. A shipped agent is scored on the test split in a separate store.

Protocol and statistics. Hypotheses, splits, budgets, seeds and analysis were fixed before the main runs; three deviations made before any τ^2-bench result existed (a lower budget, a reduced scope, and a bug fix in the ACE port) are listed in Appendix A. Each method has three seeds per benchmark, and the starting agent is scored three times. Intervals are 95% hierarchical bootstraps that resample test tasks and then each method's seeds (Efron and Tibshirani, 1993). Comparisons of CLADE with the four competitors are Holm-corrected within each benchmark (Holm, 1979).

5 Results

No optimizer reliably improves the agent on τ^2-bench. Table 1 gives the scores. On airline the methods span 0.425 to 0.479 against 0.433 for the starting agent, and none differs significantly from it or from CLADE (Table 2). On retail, only CLADE is above the starting agent. Pooled over both domains, CLADE's gain over the starting agent is +0.05 [−0.01, +0.11] ($p = 0.062$). The scale of these differences should be read against scoring noise: across all scorings of the *unchanged* starting agent, including runs that shipped it unchanged, its test score ranged from 0.38 to 0.54 on airline and from 0.44 to 0.54 on retail (Figure 1). GEPA and MIPROv2 shipped the unchanged starting prompt in all three airline runs, so their airline scores (0.467 and 0.479) are re-scorings of the starting agent. Airline is hard for this agent: on the 13 test tasks that need a database write, every method scores between 0.17 and 0.28 (Appendix E).

Optimizers often ship a worse agent. We call a run harmful when its shipped agent scores below *every* scoring of the unchanged starting agent on the same benchmark. Out of nine runs per method (three seeds on three benchmarks), GEPA had 4/9 harmful runs, GEPA+tools 4/9, MIPROv2 2/9, ACE 1/9 and CLADE 0/9. The worst shipped agents scored 0.338 on retail (against 0.44 to 0.54 for the starting agent) and 0.335 on the support desk (against 0.74 to 0.76). The overfitting gap (validation score of the shipped bundle minus its test score; Appendix Table 5) shows why: on retail and the support desk, GEPA's and GEPA+tools' shipped prompts scored 0.17 to 0.36 higher on their validation tasks than on test, CLADE's 0.02 and 0.12. One MIPROv2 prompt on the support desk addressed only the single customer who appears in the practice tasks and quoted task identifiers (Appendix F). Giving GEPA the tool descriptions as well (GEPA+tools) did not help: a larger search space with the same selection overfit further. The gap is only weakly associated with the number of candidates a run compared on its validation tasks (Spearman $\rho = 0.28$ over 45 runs), so multiple comparisons alone do not explain it; what a candidate changes matters too.

What the harmful prompts contain. GEPA's worst retail prompt (test score 0.338) rewrote τ^2-bench's two-paragraph instruction into a 6,000-character prompt whose first rule reads: "on every turn you must output exactly one of the following JSON objects and nothing else", followed by templates such as `{"type": "tool", "name": ..., "arguments": ...}`. It reinterprets τ^2-bench's request to "generate valid JSON" as an instruction to write tool calls as text, which conflicts with the model's native tool calling; the agent then attempted calls to a non-existent tool named `json`. Such rewrites change how the agent interacts, not what it knows, and a few validation tasks cannot reveal the damage. ACE's failure mode is different: its playbooks grew to between 15,876 and 64,636 characters on τ^2-bench, about 50 to 210 times the starting instruction, a cost every later episode pays, for no gain on test. CLADE's shipped prompts stayed between 306 and 710 characters.

CLADE against the competitors. Table 2 compares CLADE with each competitor. On retail CLADE is ahead of all four: significantly against GEPA+tools (+0.12 [+0.03, +0.22]) after Holm correction, marginally against MIPROv2 (+0.08 [+0.01, +0.15]; its Holm-adjusted p ranges from 0.037 to 0.050 across ten bootstrap seeds), and with $p \approx 0.07$ against GEPA and ACE. On airline all differences are within ± 0.05 and none is significant. On the development benchmark CLADE is ahead of GEPA (+0.24 [+0.08, +0.42]), GEPA+tools (+0.29 [+0.11, +0.47]) and ACE (+0.16 [+0.03, +0.33]), all significant, and of MIPROv2 by +0.18 [−0.06, +0.54], not significant because one MIPROv2 seed collapsed.

What CLADE changed. Each CLADE run on τ^2-bench evaluated eight candidates, and the selector rejected most: CLADE shipped the unchanged agent in two of three airline runs and one of three

Method	τ^2-bench airline		τ^2-bench retail		Support desk (dev.)		Cost per τ^2-bench run	
	score	Δ (changed)	score	Δ (changed)	score	Δ (changed)	USD	hours
Starting agent	0.433	–	0.475	–	0.748	–	–	–
CLADE	0.467	+0.033 (1/3)	0.533	+0.058 (2/3)	0.882	+0.133 (3/3)	3.31	2.5
GEPA	0.467	+0.033 (0/3)	0.438	−0.037 (3/3)	0.640	−0.108 (3/3)	4.49	6.7
GEPA+tools	0.425	−0.008 (2/3)	0.410	−0.065 (2/3)	0.587	−0.162 (3/3)	5.12	7.0
MIPROv2	0.479	+0.046 (0/3)	0.454	−0.021 (2/3)	0.702	−0.047 (3/3)	3.86	3.5
ACE	0.442	+0.008 (3/3)	0.475	+0.000 (3/3)	0.723	−0.025 (3/3)	4.68	8.2

Table 1: Test score (mean of three seeds), difference from the starting agent, and in brackets the number of runs whose shipped agent differs from the starting one. Cost is learning cost per run (optimizer calls and learning episodes, at list prices) and wall-clock learning time, averaged over the τ^2-bench runs.

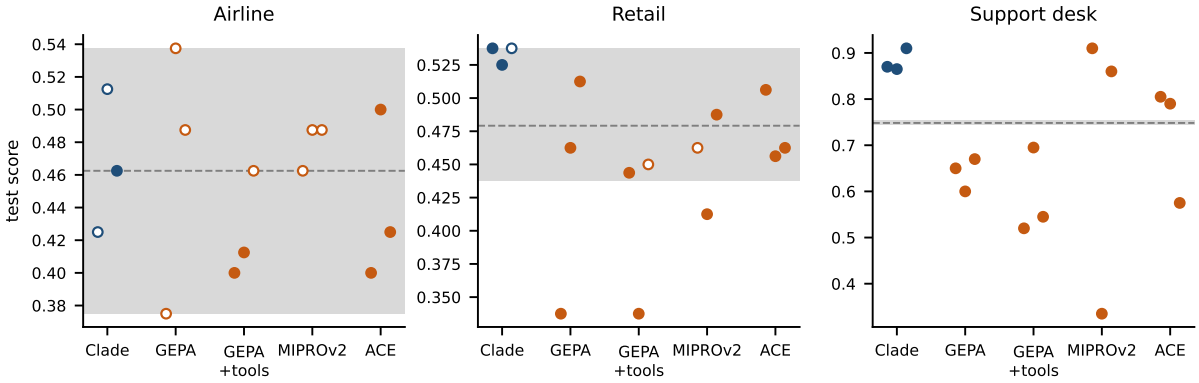


Figure 1: Every run's test score. The grey band spans all scorings of the unchanged starting agent (16 on airline, 6 on retail, 3 on the support desk) and the dashed line is their mean. Hollow markers are runs that shipped the starting agent unchanged; their spread is scoring noise.

Competitor	Airline	Retail
GEPA	+0.00 [−0.10, +0.11]	+0.10 [−0.00, +0.20]
	$p = 1.000$	$p = 0.067$
GEPA+tools	+0.04 [−0.03, +0.13]	+0.12 [+0.03, +0.22]
	$p = 0.660$	$p = 0.026$
MIPROv2	−0.01 [−0.10, +0.07]	+0.08 [+0.01, +0.15]
	$p = 1.000$	$p = 0.041$
ACE	+0.03 [−0.09, +0.15]	+0.06 [−0.01, +0.12]
	$p = 1.000$	$p = 0.067$

Table 2: CLADE minus competitor on τ^2-bench test score, with 95% intervals and Holm-adjusted one-sided p (below). Positive favours CLADE. Pooled results are in Appendix C.

retail runs. In three of the six runs the candidate with the best held-out mean was blocked by the per-task gate because one held-out task dropped from always passing to always failing. Each changed bundle added a single prompt section, for example, on retail, "After confirming the user's identity, you must invoke the appropriate domain-specific tool (exchange, cancel, modify, return) before ending the turn. Do not end the conversation or transfer to a human unless the tool call fails or the user explicitly asks." The two changed retail bundles scored 0.537 and 0.525. On the support desk all three runs changed the agent and all three improved it (0.865 to 0.910 against 0.74 to 0.76). Its protection on τ^2-bench is therefore mostly a matter of not shipping changes that do not clearly win, rather than of finding better prompts.

Cost. CLADE was the cheapest and fastest method per run (Table 1). Its critic discards fixes before they cost episodes, and each candidate's episodes run in parallel. It also used fewer rollouts: one CLADE evaluation costs 60 (30 tasks, twice each), and its budget rule starts an evaluation only with room to spare, so every τ^2-bench run stopped after nine evaluations, at 540 of 600 rollouts; ACE stopped after five epochs (313 to 350). GEPA and ACE took 6.7 and 8.2 hours per τ^2-bench run because they evaluate few episodes at a time: GEPA a minibatch of three, ACE one task at a time with reflection between attempts.

CLADE variant	score	Δ vs full	changed
full	0.467	–	1/3
rewrites	0.512	+0.05 [−0.03, +0.13]	2/3
no critic	0.438	−0.03 [−0.13, +0.05]	1/3
greedy	0.450	−0.02 [−0.11, +0.07]	2/3
no gate	0.454	−0.01 [−0.10, +0.08]	3/3

Table 3: Ablations on τ^2-bench airline, three seeds each: *rewrites* replaces bounded edits with whole-prompt rewrites; *greedy* runs one lane from the incumbent, without the archive; *no gate* drops the per-task regression gate. Changed: runs whose shipped agent differs from the starting one.

Ablations. Table 3 removes one component at a time. No removal significantly changes the test score. Replacing bounded edits with whole-prompt rewrites, still behind the critic and gate, scored 0.05 higher (not significant) and changed the agent more often; removing the gate made every run ship a change without hurting on airline. With three seeds on a 20-task test split, these experiments can detect only differences of about 0.1; they do not show which component carries CLADE's protection, and the rewrite result suggests that the gate and screening, rather than the size of an edit, may matter most.

6 Discussion

The common practice of reporting one optimized prompt against one scoring of the starting prompt does not survive our noise measurements: on τ^2-bench airline, the same agent's score moved by up to 0.16 between scorings, and most optimizers' differences from it were smaller. Two practical conclusions follow. An optimizer's output should be promoted only through a held-out comparison that accounts for run-to-run variation, as CLADE's gate does; and an evaluation of an optimizer should re-score the starting agent several times, report how often the shipped agent is worse, and report cost. Within those standards, the optimizers we tested do not reliably improve a mid-size open agent on τ^2-bench at a few hundred rollouts, and a conservative optimizer is the only safe default we found.

7 Conclusion

Under one equal-budget harness, popular prompt and context optimizers frequently shipped worse versions of a tool-using agent on τ^2-bench and a development benchmark, while gains of any method on τ^2-bench stayed within scoring noise. CLADE, which proposes bounded edits, screens fixes, and promotes only through a noise-aware held-out gate, never shipped a harmful agent, led every competitor on retail and on the development benchmark, and cost the least. Its benefit on τ^2-bench is protection, not improvement. We release the harness, the protocol and every run (`https://github.com/kumar1ashish/clade-optimiser`) so that these comparisons can be extended to other agents, budgets and benchmarks.

Limitations

We test one agent model (gpt-oss-20b) and one reflection model; larger agents, or a stronger reflection model, may behave differently. Three seeds per method and 20 to 40 test tasks per domain limit statistical power: differences below about 0.1 are not detectable; on retail, two of CLADE's four comparisons fall just short of significance and a third is marginal. Bootstrap p-values carry Monte Carlo error of about ± 0.01. We use a 600-rollout budget only. The τ^2-bench user simulator and retail judge are gpt-oss-120b rather than the gpt-4.1 used on the public leaderboard, so absolute scores are not comparable with it. Competitors run at their defaults sized to the budget, not tuned per benchmark; ACE is our port of the official code, which asks the agent to cite playbook bullets so they can be tagged. The support desk was written by us and CLADE was developed on it; its task identifiers are descriptive, every method could read them in the episode rendering (and MIPROv2 also in its example fields), and one MIPROv2 prompt quoted them. The protocol changed three times before any τ^2-bench result existed (Appendix A). Costs are at list prices without cache discounts. Finally, CLADE's main effect on τ^2-bench was to decline changes; our results support its use as a safe default, not as a method that improves this agent on this benchmark.

Ethics Statement

The study uses simulated customer-service environments without personal data or human subjects. The total compute for all reported runs was about 212 USD at list prices. Automatically optimized prompts can make deployed agents worse; our results argue for gating such changes, and we release all code and runs to make that check routine.

Acknowledgments

The author designed the method and the study, directed the experiments and the analysis, and reviewed, modified and verified the code and the manuscript. An AI assistant (Claude, Anthropic) was used as a tool to generate code, and to draft text and improve its readability, under the author's direction. The author takes full responsibility for the content.

References

Lakshya A. Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J. Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. 2026. GEPA: Reflective prompt evolution can outperform reinforcement learning. In *The Fourteenth International Conference on Learning Representations (ICLR 2026)*.

Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. 2026. τ^2-bench: Evaluating conversational agents in a dual-control environment. In *Proceedings of the 43rd International Conference on Machine Learning (ICML 2026)*, volume 306 of *Proceedings of Machine Learning Research*, pages 6736–6782. PMLR.

Ching-An Cheng, Allen Nie, and Adith Swaminathan. 2024. Trace is the next AutoDiff: Generative optimization with rich feedback, execution traces, and LLMs. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, volume 37, pages 71596–71642. Curran Associates, Inc.

DSPy. 2025. SIMBA (Stochastic Introspective Mini-Batch Ascent) optimizer, DSPy v2.6.15. Software. `https://dspy.ai/api/optimizers/SIMBA/`. Released in DSPy v2.6.15, March 2025. Source code: `https://github.com/stanfordnlp/dspy`.

Bradley Efron and Robert J. Tibshirani. 1993. *An Introduction to the Bootstrap*. Number 57 in Monographs on Statistics and Applied Probability. Chapman & Hall, New York.

Chrisantha Fernando, Dylan Sunil Banarse, Henryk Michalewski, Simon Osindero, and Tim Rocktäschel. 2024. Promptbreeder: Self-referential self-improvement via prompt evolution. In *Proceedings of the 41st International Conference on Machine Learning (ICML 2024)*, volume 235 of *Proceedings of Machine Learning Research*, pages 13481–13544. PMLR.

Lucheng Fu, Ye Yu, Yiyang Wang, Yiqiao Jin, Haibo Jin, B. Aditya Prakash, and Haohan Wang. 2026. TextReg: Mitigating prompt distributional overfitting via regularized text-space optimization. *arXiv preprint arXiv:2605.21318*.

Sture Holm. 1979. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70.

Sayash Kapoor, Benedikt Stroebl, Zachary S. Siegel, Nitya Nadgir, and Arvind Narayanan. 2025. AI agents that matter. *Transactions on Machine Learning Research*.

Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. 2024. DSPy: Compiling declarative language model calls into state-of-the-art pipelines. In *The Twelfth International Conference on Learning Representations (ICLR 2024)*.

Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. 2026. Meta-Harness: End-to-end optimization of model harnesses. In *Third Conference on Language Modeling (COLM 2026)*.

OpenAI. 2025. gpt-oss-120b & gpt-oss-20b model card. *arXiv preprint arXiv:2508.10925*.

Krista Opsahl-Ong, Michael J. Ryan, Josh Purtell, David Broman, Christopher Potts, Matei Zaharia, and Omar Khattab. 2024. Optimizing instructions and demonstrations for multi-stage language model programs. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP 2024)*, pages 9340–9366, Miami, Florida, USA. Association for Computational Linguistics.

Siru Ouyang, Jun Yan, I-Hung Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long T. Le, Samira Daruki, Xiangru Tang, Vishy Tirumalashetty, George Lee, Mahsan Rofouei, Hangfei Lin, Jiawei Han, Chen-Yu Lee, and Tomas Pfister. 2026. ReasoningBank: Scaling agent self-evolving with reasoning memory. In *The Fourteenth International Conference on Learning Representations (ICLR 2026)*.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, volume 36, pages 8634–8652. Curran Associates, Inc.

Mirac Suzgun, Mert Yuksekgonul, Federico Bianchi, Dan Jurafsky, and James Zou. 2026. Dynamic Cheatsheet: Test-time learning with adaptive memory. In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers) (EACL 2026)*, pages 7080–7106, Rabat, Morocco. Association for Computational Linguistics.

Wenyi Wang, Piotr Piękos, Li Nanbo, Firas Laakom, Yimeng Chen, Mateusz Ostaszewski, Mingchen Zhuge, and Jürgen Schmidhuber. 2026. Huxley-Gödel Machine: Human-level coding agent development by

an approximation of the optimal self-improving machine. In *The Fourteenth International Conference on Learning Representations (ICLR 2026)*.

Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. 2025. Agent workflow memory. In *Proceedings of the 42nd International Conference on Machine Learning (ICML 2025)*, volume 267 of *Proceedings of Machine Learning Research*, pages 63897–63911. PMLR.

Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. 2024. Large language models as optimizers. In *The Twelfth International Conference on Learning Representations (ICLR 2024)*.

Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Pan Lu, Zhi Huang, Carlos Guestrin, and James Zou. 2025. Optimizing generative AI by backpropagating language model feedback. *Nature*, 639(8055):609–616.

Jenny Zhang, Shengran Hu, Cong Lu, Robert Tjarko Lange, and Jeff Clune. 2026a. Darwin Gödel Machine: Open-ended evolution of self-improving agents. In *The Fourteenth International Conference on Learning Representations (ICLR 2026)*.

Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Zou, and Kunle Olukotun. 2026b. Agentic context engineering: Evolving contexts for self-improving language models. In *The Fourteenth International Conference on Learning Representations (ICLR 2026)*.

Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2024. ExpeL: LLM agents are experiential learners. In *Proceedings of the 38th AAAI Conference on Artificial Intelligence (AAAI 2024)*, volume 38, pages 19632–19642. AAAI Press.

Cen Mia Zhao, Haibo Ruan, Wenjie Chen, Pei-fen Tu, Usman Abbasi, and Joel Hesch. 2026. Beyond prompts: Measuring and optimizing LLM tool-agent harnesses. *arXiv preprint arXiv:2609.05736*. Accepted to EMNLP 2026.

Yongchao Zhou, Andrei Ioan Muresanu, Ziwen Han, Keiran Paster, Silviu Pitis, Harris Chan, and Jimmy Ba. 2023. Large language models are human-level prompt engineers. In *The Eleventh International Conference on Learning Representations (ICLR 2023)*.

A Protocol and deviations

The protocol (released with the code) fixed hypotheses, benchmarks, splits, models, methods, budgets, seeds and analysis before the main runs. Short end-to-end checks of the harness (60 to 240 rollouts) were run before it was fixed; they found only infrastructure faults and their scores were not used. Three deviations followed, each before any τ^2-bench learning run had finished: (1) the τ^2-bench budget was lowered from 900 to 600 rollouts for every method, because provider capacity put the original grid at about 1.5 days; the runs then in progress were stopped and discarded. (2) The scope was reduced from seven competitors, five seeds and ten ablations to four competitors, three seeds and four ablations on airline; SIMBA, TextGrad and MIPROv2 with demonstrations remain runnable in the release. (3) A bug in our ACE port (a section-name mismatch that crashed a run when the curator added a bullet to one section) was fixed, and the two ACE runs started before the fix were restarted from scratch. Test episodes lost to the provider after every retry (13 of all test episodes) were re-run as the protocol requires; all 13 scored as before. The original hypothesis that CLADE beats each competitor on both τ^2-bench domains is supported only on retail: clearly against one of four competitors and marginally against a second.

B Configurations

CLADE: three rounds at most, $n = 2$ runs per task in every evaluation, up to three playbook candidates per lane, three lanes, patience two rounds, critic and lenses on, playbook cap eight rules, prompt sections of at most six lines, tool descriptions of at most 400 characters, gate $\delta = 0.34$, exploration $c = 0.3$, reflection temperature 0.4. **GEPA**: library defaults with reflection minibatch 3, Pareto candidate selection, round-robin component selection, validation tasks each twice, budget passed as `max_metric_calls`; it may finish one validation pass beyond the budget. **MIPROv2**: no bootstrapped or labelled demonstrations, number of trials the largest that fits the budget (minibatches of 10 validation instances when there are more than 10, full evaluation every 5 trials), 12 instruction candidates and 40 trials per τ^2-bench run (64 on the support desk), proposal temperature 1.0, program-aware proposer off. **ACE**: official no-ground-truth reflector and curator prompts, temperature 0, batch size 1, up to three reflection rounds per failed task, up to five epochs, validation after each epoch with the best epoch shipped, curator operations ADD only, no de-duplication (the official defaults). All methods read episodes rendered as in CLADE's

reflector, at most 2,500 characters each.

C Pooled comparison

Table 4 pools the two τ^2-bench domains, weighting each equally, with Holm correction over the four competitors.

Competitor	CLADE − competitor	Holm p
GEPA	+0.05 [−0.02, +0.13]	0.315
GEPA+tools	+0.08 [+0.02, +0.15]	0.022
MIPROv2	+0.03 [−0.02, +0.09]	0.315
ACE	+0.04 [−0.03, +0.11]	0.315

Table 4: CLADE minus competitor pooled over τ^2-bench airline and retail, each domain weighted equally.

D Per-seed results

Table 5 lists every run.

Bench.	Method	test score per seed	gap	rollouts
Airline	CLADE	0.425° 0.512° 0.463	+0.13	540
Airline	GEPA	0.375° 0.537° 0.487°	+0.08	608
Airline	GEPA+tools	0.400 0.412 0.463°	+0.19	608
Airline	MIPROv2	0.463° 0.487° 0.487°	+0.06	600
Airline	ACE	0.400 0.500 0.425	+0.15	313
Retail	CLADE	0.537 0.525 0.537°	+0.02	540
Retail	GEPA	0.338 0.463 0.512	+0.19	619
Retail	GEPA+tools	0.444 0.338 0.450°	+0.17	611
Retail	MIPROv2	0.463° 0.412 0.487	+0.01	600
Retail	ACE	0.506 0.456 0.463	+0.04	350
Support desk	CLADE	0.870 0.865 0.910	+0.12	333
Support desk	GEPA	0.650 0.600 0.670	+0.30	404
Support desk	GEPA+tools	0.520 0.695 0.545	+0.36	402
Support desk	MIPROv2	0.910 0.335 0.860	+0.28	400
Support desk	ACE	0.805 0.790 0.575	−0.00	93

Table 5: Test score of each seed (°: shipped the starting agent unchanged), overfitting gap (validation score of the shipped bundle minus test score, mean over seeds) and learning rollouts used.

E Prompt size and write tasks

Table 6 gives the mean length of each method's shipped instruction, and its test score on the τ^2-bench tasks whose expected outcome needs a database write. On those tasks every method stays close to the starting agent; most of airline's score comes from tasks an agent passes by taking no harmful action.

	shipped prompt (characters)			write tasks	
Method	airline	retail	support	airline	retail
Starting agent	306	306	257	0.212	0.431
CLADE	435	575	824	0.237	0.488
GEPA	306	3,555	7,077	0.237	0.382
GEPA+tools	306	377	5,250	0.212	0.356
MIPROv2	306	2,067	4,972	0.276	0.410
ACE	41,788	32,575	4,066	0.173	0.428

Table 6: Mean length of the shipped instruction including learned rules, and mean test score on the τ^2-bench test tasks whose expected outcome needs a database write (13 of 20 on airline, 36 of 40 on retail). An agent that takes no action passes the remaining tasks.

F Examples of shipped bundles

CLADE, τ^2-bench retail, seed 1 (one prompt section added to τ^2-bench's default instruction):

> **Missing Information Handling.** If any required detail (e.g., order ID, purchase date, last-4 of payment) is absent, first ask the user for that specific info. Do not transfer to a human until the missing data has been obtained and the appropriate tool call has been executed. After receiving the data, immediately invoke the relevant lookup or update tool before sending any reply.

MIPROv2, support desk, seed 1 (test score 0.335; the opening of the shipped instruction):

> You are an autonomous customer-support assistant for Northwind Outfitters, handling only emails from the single customer Leo Garcia [...] order IDs are always of the form O200X [...] look for keywords that map to one of the fine-grained task IDs (e.g., t-opened-not-damaged, t-refund-unopened, [...]).

G Reproducibility

One command runs each grid and one generates every table and figure in this paper from the released run files. All runs use gpt-oss-20b and gpt-oss-120b served by Groq. The release pins τ^2-bench at v1.0.1 and the ACE prompts at the official repository's commit used here.